LAB 2: Python Data Structures and File Handling

Procedure

Step 1:

Concept Overview:
The table compares Lists, Tuples, Sets, and Dictionaries based on ordering, indexing, mutability, duplicates, and data types.

Key Detail:
Lists and Tuples are ordered and indexed, while Sets are unordered and do not allow duplicates.

Insight:
Dictionaries store data as key-value pairs instead of index-based access.

LISTS VS. TUPLES VS. SETS VS. DICTIONARIES

	Lists	Tuples	Sets	Dictionaries
Ordering	Ordered	Ordered	Unordered	Ordered Unordered before Python 3.7
Indexing	Indexed	Indexed	Not Indexed	Keyed
Mutability	Mutable	Immutable	Mutable Only Adding and Removing	Mutable
Duplicates Allowed	Yes	Yes	No	Yes Only in values and not in keys
Types Allowed	Mutable and Immutable	Mutable and Immutable	Only Immutable	Only Immutable In keys

Figure 2.1

Step 2:

Structure Hierarchy:
The diagram shows Python Data Structures divided into built-in and user-defined types.

Built-in Types:
List, Dictionary, Tuple, and Set are shown as core structures.

Extended Structures:
Stack, Queue, Trees, Linked Lists, Graphs, and HashMaps are derived structures.

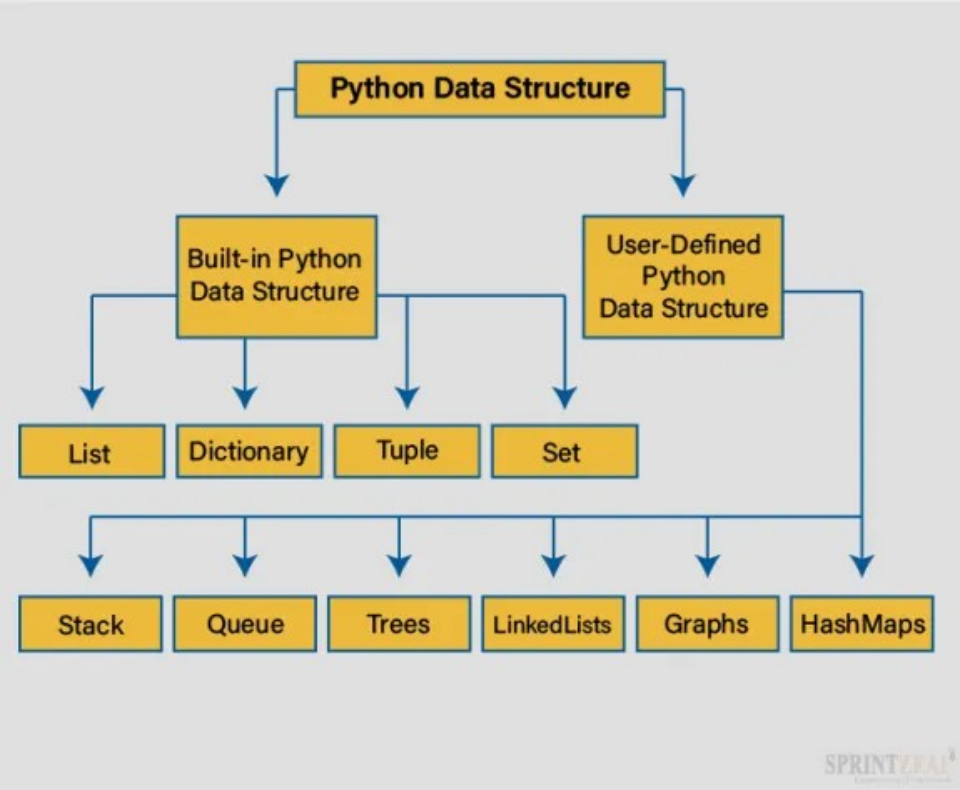

Figure 2.2

Step 3:

List Comprehension:
The syntax [x+1 for x in range(5) if x%2 == 2] is displayed.

Components:
It shows output expression, collection, and condition sections.

Purpose:
This demonstrates creating lists efficiently using a single line of code.

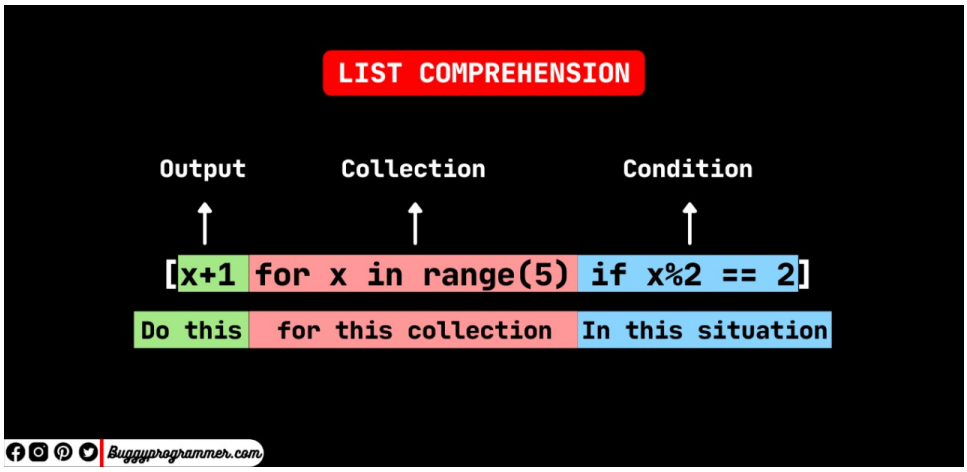

Figure 2.3

Step 4:

Flowchart Logic:
The diagram shows a loop processing numbers from a list.

Condition Check:
Each number is checked and conditionally added to a new list.

Flow:
The process continues until all elements are processed.

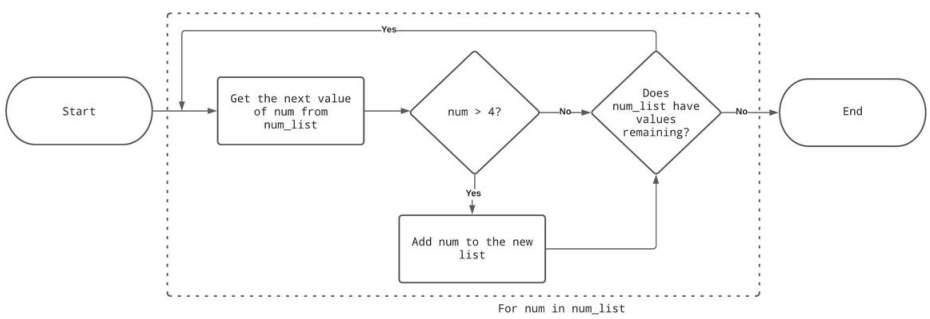

Figure 2.4

Step 5:

Data Structures:
A list of students ['Ali','Sara','Ahmed'], a tuple of grades (85,90,78), a dictionary mapping names to marks, and a set of subjects are created.

Output:
The printed result shows the list and dictionary values.

```python
# List
students = ["Ali", "Sara", "Ahmed"]

# Tuple
grades = (85, 90, 78)

# Dictionary
student_data = {
    "Ali": 85,
    "Sara": 90,
    "Ahmed": 78
}

# Set
subjects = {"Math", "Physics", "Chemistry"}

print(students)
print(student_data)
```

```
['Ali', 'Sara', 'Ahmed']
{'Ali': 85, 'Sara': 90, 'Ahmed': 78}
```

Figure 2.5

Step 6:

Function Definition:
A function calculate_average computes sum(grades)/len(grades).

Execution:
Grades [85,90,78] produce average 84.33.

Result:
The output displays 'Average grade: 84.33'.

```python
def calculate_average(grades):
    return sum(grades) / len(grades)

grades = [85, 90, 78]
average = calculate_average(grades)

print("Average grade:", average)
```

```
Average grade: 84.33333333333333
```

Figure 2.6

Step 7:

CSV Reading:
The csv module reads 'students.csv'.

Loop:
Each row prints Name and Marks.

Output:
Ali 85, Sara 90, Ahmed 78 are displayed.

```python
import csv

with open("data/students.csv", "r") as file:
    reader = csv.DictReader(file)

    for row in reader:
        print(row["Name"], row["Marks"])
```

```
Ali 85
Sara 90
Ahmed 78
```

Figure 2.7

Step 8:

JSON Writing:
A dictionary of student marks is written to 'students.json' using json.dump().

Purpose:
This converts Python data into JSON format.

```python
import json

student_data = {
    "Ali": 85,
    "Sara": 90,
    "Ahmed": 78
}

with open("students.json", "w") as file:
    json.dump(student_data, file)
```

Figure 2.8

Step 9:

File Reading:
A text file 'notes.txt' is opened and its content is printed.

Output:
The displayed string confirms file reading operation.

```python
with open("notes.txt", "r") as file:
    content = file.read()

print(content)
```

```
wef7488erf834r8ud348rud9438
```

Figure 2.9

Step 10:

List Comprehension Execution:
Numbers [1,2,3,4,5] are squared using [n**2 for n in numbers].

Result:
Output is [1,4,9,16,25].

```python
numbers = [1,2,3,4,5]

squares = [n**2 for n in numbers]

print(squares)
```

```
[1, 4, 9, 16, 25]
```

Figure 2.10

Step 11:

Exception Handling:
User input is divided into 10.

Errors:
ZeroDivisionError and ValueError are handled.

Final:
'Program finished' is always printed.

```python
try:
    num = int(input("Enter number: "))
    result = 10 / num
    print(result)

except ZeroDivisionError:
    print("Cannot divide by zero")

except ValueError:
    print("Invalid input")

finally:
    print("Program finished")
```

```
2.0
Program finished
```

Figure 2.11

Step 12:

CSV Calculation:
Sales data is read and total calculated using Price * Quantity.

Result:
Total Sales is printed as 7200.

```python
import csv

total_sales = 0

with open("data/sales.csv") as file:
    reader = csv.DictReader(file)

    for row in reader:
        total = int(row["Price"]) * int(row["Quantity"])
        total_sales += total

print("Total Sales:", total_sales)
```

```
Total Sales: 7200
```

Figure 2.12